

USE

Universal StoragE

대학 자산 통합 운영 MSA 플랫폼

학과·연구실·동아리의 자산 대여·반납·도입 업무를 서비스별 책임으로 분리하고, REST와 Kafka 계약으로 연결한 B2B 자산 운영 플랫폼입니다. 수요 분석은 핵심 트랜잭션과 분리된 독립 서비스로 구성했습니다.



이해관계자와 Pain Point

기능과 화면 모두에서 끊어진 운영 흐름

1

일반 구성원

하는 일

동아리, 학생회 장비를 빌리고 반납

PAIN POINT

어떤 자산을 빌릴 수 있는지, 신청 상태가 어떻게 되었는지 확인하기 어려움

제공 가치

속한 그룹의 자산을 한 화면에서 확인 및 신청한다.

2

그룹 관리자

하는 일

그룹 자산과 신청을 운영

PAIN POINT

승인, 반납, 재고 정보를 따로 확인 및 수기로 관리함

제공 가치

요청 승인과 재고 변화를 하나의 업무 흐름으로 관리한다.

3

학교 관리자

하는 일

전체 예산과 공용 자산을 운영

PAIN POINT

예산 선정에 대한 정보 부족

제공 가치

다음 4주 수요를 참고해서 예산을 분배에 도움을 받을 수 있다.

제품 목표와 핵심 흐름

인증부터 대여·반납·도입까지 하나의 업무 흐름으로

사용자 요청과 관리자 승인을 연계하되, 회원·재고·요청·예산의 책임을 서비스별로 분리해 운영

1

로그인·그룹 참여

2

자산 조회

3

대여·반납

4

미보유 장비
도입

5

수요
분석

Vue Router 기반 인증 진입부터 그룹별 동적 워크스페이스까지 연결하고, 학교 공용 자산과 그룹 자산을 하나의 UI 체계에서 운영하도록 설계했습니다.

학교 공용 자산

그룹 자산

하나의 동적 워크스페이스

기능 추가보다 먼저, 서비스 책임을 다시 나눴습니다

각 서비스가 자기 데이터와 상태를 소유하도록 대학 자산 운영 도메인에 맞게 경계를 재정의

Member	Asset	Request	Budget	Demand Analytics
사용자 · 그룹 · 권한	자산 · 재고 · 입고	대여 · 반납 · 도입 상태	학교 예산 검토	이벤트 · 학습 · 예측
users / campus_groups group_memberships	courses	enrollments	payments	analytics_*

① 데이터 소유권
다른 서비스 DB를 직접 수정하지 않음

② 상태 책임
Request가 업무 상태, Asset이 재고를 책임

③ 변경 영향 격리
예산·분석 확장이 핵심 대여 서비스와 분리

‘어떤 서비스가 어떤 책임과 데이터를 소유하는가’를 기준으로 확장

서비스 간 소통 방식을 REST와 Kafka로 구분했습니다

사용자 응답에 필요한 협력은 동기 호출, 핵심 트랜잭션과 분리 가능한 후속 처리는 이벤트로 연결

Vue Web → API Gateway

Member

사용자 · 그룹 · 권한

Asset

자산 · 재고 · 입고

Request

대여 · 반납 · 도입 상태

Budget

학교 예산 검토

Demand Analytics

이벤트 · 학습 · 예측

동기 REST

Request → Member 권한 확인 · Request → Asset 재고 차감/복구 · Request → Budget 예산 요청

비동기 Kafka

Budget → payment.completed → Request
Request → rental.lifecycle → Demand Analytics

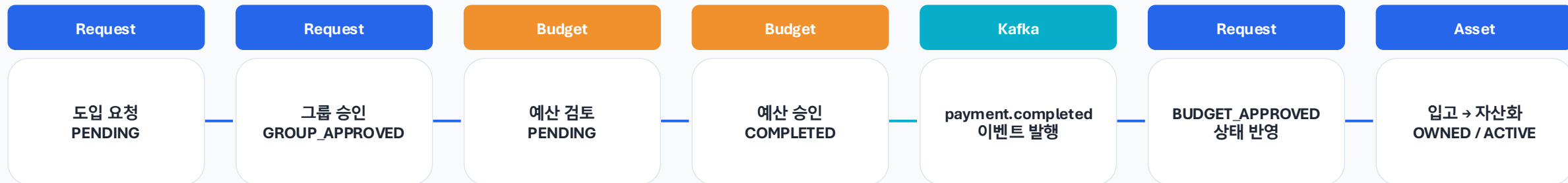
통신 방식 선택 기준

REST = 승인 결과·재고
처럼 즉시 확인 필요
Kafka = 예산 완료·분석
처럼 후속 처리 가능

서비스 간 DB를 공유하지 않고 API·이벤트 계약으로만 협력 → 데이터 소유권과 독립 변경 가능성을 유지

하나의 요청을 서비스별 로컬 트랜잭션으로 완성합니다

도입 요청을 예로, 각 서비스가 자신의 상태만 변경하고 완료 결과를 이벤트로 다음 서비스에 전달



서비스별 트랜잭션

각 서비스 내부 변경은 로컬 트랜잭션으로 처리

서비스 간 상태 전파

예산 완료 결과는 Kafka 이벤트로 Request에 전달

현재 한계

분산 트랜잭션 실패 시 보상 Saga / Outbox는 미구현

핵심은 하나의 DB 트랜잭션이 아니라, 서비스별 상태를 API·이벤트 계약으로 연결하는 것

서비스 간 협력을 API 계약으로 명세했습니다

화면용 외부 API와 서비스 간 내부 API를 구분해 데이터 소유권을 침범하지 않는 호출 방향을 정의

Service	Method	Endpoint	목적	MSA 관점 계약
Member	GET	/api/members/me	현재 사용자 정보 조회	JWT 사용자 식별, groupRole 반환
Member	GET	/internal/groups/{groupId}/members/{userId}	그룹 권한 확인	Request가 승인·반납 처리 전 호출
Asset	GET	/api/assets?groupId=&scope=	공용/그룹 자산 조회	availableQuantity 포함, 화면 목록 제공
Asset	PATCH	/internal/assets/{assetId}/stock	재고 차감·복구	대여 승인/반납 확인 시 Request가 호출
Request	POST	/api/requests/rentals	대여 신청 생성	PENDING 상태 생성, 신청자·수량 저장
Request	PATCH	/api/requests/rentals/{id}/approve	대여 승인	권한 확인 → 재고 차감 → APPROVED
Request	PATCH	/api/requests/rentals/{id}/return	반납 확인	재고 복구 → RETURNED
Budget	PATCH	/api/budgets/{id}/approve	예산 승인	승인 완료 후 payment.completed 발행
Analytics	GET	/api/analytics/demand?groupId=&weeks=4	4주 수요예측 조회	부족 수량·이동 제안 반환

핵심 업무 흐름은 API 호출과 이벤트로 연결됩니다

대여 승인·도입 요청·수요예측 흐름에서 어떤 서비스가 동기/비동기로 협력하는지 명세화

대여 승인 Flow

- 1. PATCH /api/requests/rentals/{id}/approve
- 2. Request → Member 권한 확인
- 3. Request → Asset 재고 차감
- 4. Request 상태 APPROVED
- 5. rental.lifecycle 이벤트 발행

반납 확인 Flow

- 1. PATCH /api/requests/rentals/{id}/return
- 2. Request → Asset 재고 복구
- 3. Request 상태 RETURNED
- 4. rental.lifecycle 이벤트 발행
- 5. Analytics 학습 데이터 축적

도입·예산 Flow

- 1. POST /api/requests/introductions
- 2. 그룹 관리자 승인
- 3. Budget 예산 검토/승인
- 4. payment.completed 이벤트 발행
- 5. Request → Asset 입고·자산화

Event	Producer	Consumer	Payload 핵심 필드	발행 시점
rental.lifecycle	Request	Analytics	requestId, assetId, groupId, quantity, status, occurredAt	대여 승인/반납 완료
payment.completed	Budget	Request	budgetId, introductionRequestId, result, approvedAmount	학교 예산 승인/반려
asset.stock.changed	Asset	Analytics	assetId, groupId, delta, availableQuantity, reason	재고 차감/복구/입고

검증을 거쳐 MSA 책임과 통신 방식을 구체화했습니다

기능을 늘리기보다 문제 → 최소 흐름 → 서비스 경계 순으로 설계를 발전

SPRINT별 설계 변화

- 1 Sprint 1
Discovery
- 2 문제·사용자
정의
- 3 Sprint 2
Walking Skeleton
- 4 기존 API로
업무 흐름 검증
- 5 Sprint 3
MSA Expansion
- 6 책임·데이터·
통신 재설계

SPRINT 3에서确定的한 MSA 설계 기준

서비스별 데이터 소유권 · 상태 책임 분리 · 즉시 결과는 REST · 후속 처리/분석은 Kafka · DB
직접 공유 금지

설계 변화

기존 Course / Enrollment / Payment 중심 구조를 대학 자산 운영 도메인의
Member / Asset / Request / Budget / Demand Analytics 책임으로 재정의

BOUNDARY

결과

대여 승인·반납은 동기 API 계약으로 즉시 반영하고, 예산 완료·대여 이력은
이벤트로 전달해 핵심 업무와 후속 처리를 분리

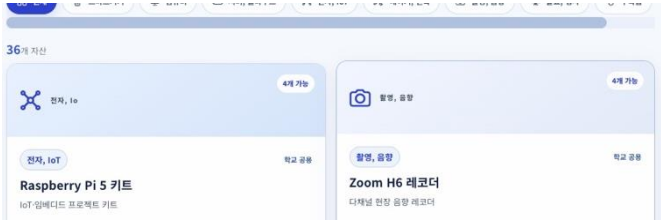
CONTRACT

Live Demo

미보유 장비 요청 → 그룹 검토 → 예산 승인 → 입고 · 자산화

3분 10초

1



자산 탐색

필요한 장비가 없다면 미보유 장비 도입 요청

2



도입 요청

장비·수량·예산과 도입 필요성을 작성

3



그룹 검토

현재 재고와 요청 근거를 확인해 필요성 승인

4



예산 승인

학교 관리자가 구매 비용을 검토하고 예산 승인

5



입고 · 자산화

입고 수량과 보관 장소 등록 후 자산으로 전환

6



요청 완료

요청부터 그룹·예산 검토, 자산화까지 상태 확인

검증 · 회고 · 한계

Done의 근거와 솔직한 한계

KEEP

- Walking Skeleton + 공통 Frontend 디자인 기준
- 운영 / 분석 데이터 분리
- 기준선 · 시간순 평가 · 오차 공개

PROBLEM

- 서비스 간 분산 트랜잭션 보상 미구현
- 분석 서비스가 합성 대여 이력 기반
- 전·후 상태와 Frontend 개선 화면 캡처 보완 필요

TRY

- 실제 LIVE 데이터 월별 WAPE 추적
- Outbox · Saga · 내부 서비스 인증 Spike
- 인증·대여·반납·분석 화면의 전후 캡처 추가

팀 소개

조원 역할과 담당



문관록

Development Team



김광현

Development Team



이민형

Development Team



임유리

Development Team



김도현

Product Owner



김선정

Scrum Master